

Documento de Especificación Técnica: Sistema Inteligente de Gestión de Asistencia e Intervención Académica

Curso: Automatización Inteligente de Procesos **Institución de Aplicación:** Colegio Rafael Narváez Cadenillas (Trujillo) **Tecnologías Core:** AWS Rekognition, Google Antigravity Framework, Claude Code, Docker Compose, PostgreSQL y Base de Datos Vectorial.

1. Introducción y Enfoque del Problema

El núcleo inicial de nuestra propuesta contemplaba únicamente el uso de **AWS Rekognition** para el registro facial de la asistencia escolar. Sin embargo, tras analizar el proceso de manera estricta, determinamos dos factores críticos:

1. El reconocimiento facial y su inserción en una base de datos relacional es un flujo determinista y lineal; no requiere el uso de Inteligencia Artificial agéntica o modelos de lenguaje (LLMs).
2. Dejar el proyecto en ese estado causaría una penalización crítica en el **Criterio 4 (Complejidad)** de la rúbrica, el cual sanciona los flujos lineales o aquellos que puedan resolverse con código tradicional sin aportar valor real mediante arquitecturas multiagente.

La Solución Agéntica

Mantendremos el módulo de visión computacional (AWS Rekognition) como un sensor pasivo de captura de datos crudos. La verdadera automatización inteligente y la arquitectura multiagente se activarán **inmediatamente después del cierre de la marcación**, abstrayendo el procesamiento de datos no estructurados (excusas de padres de familia), el análisis predictivo de deserción escolar y la resolución de conflictos institucionales.

2. Arquitectura de Despliegue del Ecosistema (docker-compose)

Para garantizar que el sistema sea completamente funcional, seguro y desplegable en un servidor VPS real mediante variables de entorno y API keys (protegiendo las credenciales de los modelos de IA), estructuraremos el proyecto en un entorno de microservicios con cuatro contenedores principales:

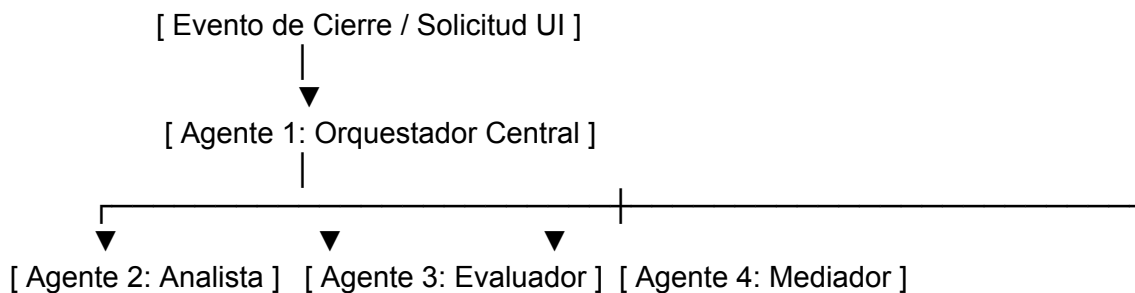
- **Contenedor UI (Frontend):** Interfaz web interactiva dividida en tres perfiles de usuario (Director/Administrador, Profesor/Tutor, y Padre de Familia).
- **Contenedor API (Backend + Swarm de Agentes):** Servidor central (en Node.js o Python) donde residirá el framework de agentes, el objeto de memoria compartida y las API keys. Expone endpoints tradicionales a la UI y coordina las llamadas de los agentes.
- **Contenedor PostgreSQL:** Base de datos relacional encargada de almacenar datos estructurados (perfiles de usuarios, registros de asistencia crudos, matrículas y agenda

de citas).

- **Contenedor Vector Database (ej. ChromaDB / Qdrant):** Almacenamiento especializado para vectores (embeddings). Contendrá el reglamento interno de asistencia del colegio fragmentado y semánticamente indexado para ejecutar técnicas de RAG (Retrieval-Augmented Generation).

3. Diseño del Swarm Multiagente (Arquitectura Antigravity)

Para cumplir con el **Criterio 1** (responsabilidades únicas, roles claramente definidos y sin solapamiento) y el **Criterio 2** (patrón de Swarms), utilizaremos la metodología de construcción que planteamos en Antigravity para programar un enjambre de cuatro entidades en nuestro backend:



1. Agente Orquestador

- **Responsabilidad:** Actúa como el ruteador del sistema y la única puerta de entrada desde la API. Inicializa el estado o memoria compartida del hilo conductor y determina a qué subagente transferir el control de la conversación (*handoff*).

2. Agente Analista Conductual (Dominio de Datos Históricos)

- **Responsabilidad:** Analiza el comportamiento histórico del estudiante. Al activarse por una falta, consulta la base de datos relacional para detectar patrones ocultos de inasistencia (por ejemplo, si el alumno suele faltar consecutivamente los días de evaluaciones de un curso específico o acumula tardanzas sistemáticas). Genera un diagnóstico de riesgo conductual o deserción.

3. Agente Evaluador de Justificaciones (Dominio de Reglas y RAG)

- **Responsabilidad:** Encargado de procesar lenguaje natural no estructurado. Cuando un padre envía una excusa por texto o adjunta un documento médico, este agente consulta de forma semántica la base de datos vectorial para extraer la normativa exacta del reglamento del colegio y juzgar de manera autónoma si la falta califica para ser justificada o no.

4. Agente Mediador y Conciliador (Dominio de Resolución de

Conflictos)

- **Responsabilidad:** Ejecuta las acciones correctivas y de comunicación externa. Si la justificación es rechazada o si el *Analista Conductual* reporta una reincidencia severa, este agente evalúa el conflicto y decide la gravedad de la sanción (ej. emitir una amonestación leve en el sistema o agendar de forma obligatoria una citación presencial con el psicólogo del colegio).

4. Flujos de Trabajo No Lineales (Líneas de Tiempo)

El sistema operará bajo un esquema desacoplado y asíncrono para manejar las dos realidades de la gestión escolar:

A. El Cierre de Asistencia y Proceso Post-Falta

1. A una hora determinada, el Director ejecuta desde la UI el "Cierre de Asistencia".
2. El sistema tradicional computa en PostgreSQL la diferencia de conjuntos mediante lógica relacional:
3. Si el alumno faltante posee un permiso pre-aprobado, Postgres lo marca automáticamente como Justificado.
4. Si no cuenta con permiso, se asienta como Inasistencia Injustificada y se dispara un evento en el **Event Bus** del backend.
5. El **Agente Orquestador** toma el caso, el **Analista Conductual** genera el diagnóstico, y el **Agente Mediador** congela la incidencia en la base de datos con el estado *Pendiente_Justificacion*, notificando al portal del padre.
6. Días u horas después, cuando el padre ingresa a la UI y redacta la justificación adjuntando la receta médica, la API "descongela" el caso. El **Agente Evaluador (RAG)** contrasta la información contra el reglamento y el Swarm emite el dictamen final actualizando la base de datos.

B. El Proceso Pre-Falta (Proactivo)

1. Con días de anticipación, el padre de familia ingresa al sistema para solicitar un permiso (ej. intervención médica programada).
2. El **Agente Evaluador** analiza la validez del motivo contra la base de datos vectorial del reglamento.
3. Si cumple con los requisitos institucionales, el **Agente Mediador** ejecuta una llamada de escritura para insertar de forma anticipada los días autorizados en la tabla de excepciones de PostgreSQL.

5. Protocolo de Comunicación e Integración (MCP)

Para cumplir estrictamente con el **Criterio 3** (uso de Model Context Protocol y gestión explícita del estado), los agentes no harán consultas directas a los motores de bases de datos. La API expondrá herramientas internas estructuradas bajo esquemas JSON validados (servidores MCP locales):

- `mcp_buscar_historial_asistencia(alumno_id)`: Devuelve el consolidado del mes desde

PostgreSQL.

- `mcp_consultar_reglamento_escolar(query_semantica)`: Transforma la cadena de texto en un embedding y devuelve las cláusulas más cercanas desde la base de datos vectorial.
- `mcp_registrar_citacion_padre(alumno_id, motivo, urgencia)`: Inserta físicamente la cita con el tutor o psicólogo en PostgreSQL.

Gestión de la Memoria Compartida

A nivel de código de backend, utilizaremos un objeto estructurado global que mutará a lo largo del flujo de ejecución de los agentes, sirviendo como el contexto unificado del Swarm sin necesidad de saturar las llamadas al LLM con historiales repetidos:

```
{
  "alumno_id": 104,
  "tipo_flujo": "Post-Falta",
  "estado_actual": "Pendiente_Evaluacion",
  "analisis_conductual": "Patrón crítico: Faltas concurrentes los días martes.",
  "clausula_reglamento_RAG": "Artículo 15: Solo se justifican inasistencias por salud con visación médica previa.",
  "dictamen_final": "Justificación rechazada por falta de sustento oficial. Se procede a agendar cita psicopedagógica por reincidencia.",
  "mcp_logs": ["mcp_buscar_historial_asistencia", "mcp_consultar_reglamento_escolar"]
}
```

6. Módulos y Requerimientos de la UI (Para el desarrollo del Frontend)

Para evitar sobreingeniería, las consultas comunes de lectura (como listar alumnos o ver asistencia histórica) se harán mediante peticiones REST tradicionales directo a PostgreSQL. La UI interactuará con los agentes de IA exclusivamente en las siguientes pantallas:

1. **Vista del Administrador/Director:** Dashboard con el botón de "Cierre de Jornada" (simulador), tabla de incidencias con estados dinámicos (Presente, Inasistencia Injustificada, Alerta Roja, Justificado) y bandeja de citaciones autónomas.
2. **Vista del Profesor/Tutor:** Visualización de alertas de riesgo de deserción emitidas por el *Agente Analista*.
3. **Portal del Padre (Módulo Agéntico):** Interfaz de chat interactivo que incluye un componente para adjuntar archivos (PDF/Imagen) para las recetas médicas, provisto de etiquetas visuales dinámicas que indiquen qué agente del Swarm está respondiendo en cada turno (ej: [IA: Evaluador de Reglamento]).

7. Matriz de Cumplimiento de la Rúbrica de Laboratorio

Para nuestra tranquilidad grupal, este diseño asegura la máxima calificación basándose en los siguientes puntos explícitos de la rúbrica:

Criterio de Evaluación	Requisito de la Rúbrica (100%)	Cómo lo cumplimos en este diseño
1. Diseño de Arquitectura Multiagente	Roles con responsabilidades únicas, sin solapamiento. Orquestador y subagentes bien diferenciados.	Definimos 4 agentes específicos con dominios cerrados (Ruta, Historial, Reglamento/RAG y Conflictos).
2. Implementación Técnica	Integración correcta de componentes (grafo de agentes, memoria compartida, event bus). Uso de Swarms.	Modelamos un patrón Swarm con pasaje de control (<i>handoff</i>), coordinado a nivel de API por eventos ante faltas y un objeto de estado explícito.
3. Comunicación y Coordinación	Uso de MCP con JSON/schema validado. Estado compartido explícitamente gestionado. Mecanismo de resolución de conflictos.	Los agentes consumen datos exclusivamente mediante funciones MCP con esquemas estrictos. El Agente Mediador resuelve las discrepancias entre las excusas del usuario y el reglamento de la Vector DB.
4. Complejidad del Caso de Estudio	Escenarios con decisiones condicionales, paralelas o iterativas. Lógica no trivial. Variabilidad significativa.	Rompimos el flujo lineal acoplando justificaciones proactivas (Pre-falta) y reactivas (Post-falta) asíncronas basadas en datos no estructurados y RAG.
5. Evaluación, Pruebas y Documentación	Métricas cuantitativas (latencia, tasa de éxito, token usage).	Nuestro contenedor de la API registrará e imprimirá de forma nativa en la consola las métricas de consumo de tokens y tiempo de respuesta tras cada corrida del Swarm.